SRS FORM

The Snake Within Game

Software Engineering

Mam Eman Munir

HAIDER JALIL MALIK-2500654

Contents

Chapter 1: Introduction

1.1 Purpose of the Document

This Software Requirements Specification (SRS) document lists all the requirements for a game called The Snake Within. The game is a modern version of the classic Snake game, made using C++ and a graphics library called SFML.

This document acts as an agreement between the developers and the course evaluators. It clearly describes what the system must do. It covers what features are needed, what limits exist, and what rules were followed during development. The goal is to guide the team through all stages of building the software.

1.2 Intended Audience

This document is written for three groups of people:

- Student Developers: They will use this document as a guide while building the game.
- Course Instructor / Evaluator: They will use it to check whether the game was built correctly and meets all the requirements.
- Open Source Community: Anyone who wants to study, copy, or improve this project can use this document to understand it.

1.3 Project Overview

The Snake Within is a Snake game for desktop computers. It is built in C++ using the SFML 3.x graphics library. The player controls a snake on a 20x20 grid and tries to eat food. Every time the snake eats food, it grows longer. The game ends when the snake hits a wall or itself, depending on the mode the player chose.

Before the game starts, the player goes through several setup screens. They choose how walls work, what color the snake is, and what type of fruit they want. After the game ends, the player can see their score and play again. All game logic is handled inside a Snake class, and the screens and states are managed in main.cpp.

1.4 Document Conventions

Word	Meaning
"shall"	The feature is required and must be included.
"should"	The feature is recommended but not required.
"may"	The feature is optional.
FR-01, FR-02…	Labels for Functional Requirements.

Word	Meaning
NFR-01, NFR-02…	Labels for Non-Functional Requirements.

Chapter 2: Software Requirements Specification

2.1 Objectives

The main goals of The Snake Within are:

1. Build a working Snake game that runs smoothly on a 20x20 grid using SFML 3.x.
2. Create a clean multi-screen flow: main menu → setup screens → ready screen → gameplay → game over.
3. Support two wall modes, four snake color themes, and four fruit types that the player can choose before playing.
4. Make the game faster as the player scores more points.
5. Remember the highest score during the session so the player can see it across multiple rounds.
6. Keep all snake movement and food logic inside the Snake class, and keep main.cpp focused only on screen management.
7. Make sure all screens, text, and effects display correctly at 60 frames per second.

2.2 Stakeholders

Stakeholder	Role	Interest / Concern
Student Developers	Build the system	Correctly build and test the game logic, graphics, and OOP design.
Course Instructor	Evaluate the work	Check if the game meets all the stated requirements.
End Users / Players	Play the game	Have a smooth, fun, and stable game with working customization options.
Open Source Community	View the code	Understand and build upon the project for learning or research.

2.3 System Context and Product Perspective

The Snake Within is a standalone desktop application. It does not need the internet, a database, or any online service. All game data is kept in memory while the game is running. The only external file the game needs is a font file called arial.ttf. On Windows, the SFML library files must be placed in the same folder as the game.

The game runs as a continuous loop. Every frame it checks for keyboard input, updates the game state on a timer, and redraws the screen. All graphics are drawn using basic shapes and text through SFML. No image files or sprite sheets are needed.

2.4 Functional Requirements

FR#	Priority	Description
FR-01	High	The game shall show a main menu screen with the title, a short subtitle, and a blinking message that says "Press ENTER to Begin."
FR-02	High	The game shall show a wall mode screen where the player picks either "Wrap Around" (Nokia style) by pressing W, or "Die on Wall" (Classic) by pressing D.
FR-03	High	The game shall show a snake color screen where the player picks one of four styles — Classic Green, Colorful Rainbow, Ocean Blue, or Sunset Orange — by pressing 1, 2, 3, or 4.
FR-04	High	The game shall show a fruit type screen where the player picks one of four options — Apple, Orange, Blueberry, or Golden — by pressing 1, 2, 3, or 4.
FR-05	High	The game shall show a summary screen before the game starts, displaying the wall mode, snake color, and fruit type the player chose.
FR-06	High	The game shall place the snake in the center of a 20x20 grid at the start, with a length of 3 segments.
FR-07	High	The snake shall move one step at a time in the chosen direction. The speed shall increase as the player scores more points.
FR-08	High	The game shall not allow the snake to turn in the opposite direction. For example, if it is moving right, pressing left shall be ignored.
FR-09	High	The game shall place food at a random empty cell. When the snake eats it, the snake grows by one segment and new food appears.
FR-10	High	The game shall end if the snake runs into itself. In Classic mode, it shall also end if the snake hits a wall.

FR#	Priority	Description
FR-11	High	In Wrap-Around mode, if the snake goes past the edge of the grid, it shall appear on the opposite side.
FR-12	High	During gameplay, the screen shall show a top bar with the current score, the session high score, and a reminder that P pauses the game.
FR-13	Medium	The player shall be able to pause and resume the game by pressing P. While paused, a dark overlay and the word "PAUSED" shall be shown on screen.
FR-14	High	When the game ends, the screen shall show the final score, the high score (or "NEW HIGH SCORE!" if beaten), and a blinking message to press ENTER to play again.
FR-15	High	Food shall be drawn as a colored circle based on the fruit type the player chose. A small white dot shall appear on it to look like a shine.
FR-16	High	The snake body shall be drawn with colors that gradually change based on the chosen theme. The head segment shall have two small white dots for eyes.

2.5 Non-Functional Requirements

NFR#	Category	Requirement
NFR-01	Performance	The game shall run at 60 frames per second. The snake shall move at a set interval that starts at 200ms and gets shorter as the score goes up.
NFR-02	Usability	All screens shall be controlled with the keyboard only. Each screen shall clearly show which keys do what.
NFR-03	Visual Quality	The game grid shall have a green checkerboard pattern. All overlays such as pause and game over shall be semi-transparent so the game is still visible behind them.
NFR-04	Portability	The code shall work on Windows using standard C++ tools and SFML 3.x. All needed files including the .exe and DLL files shall be in one folder.
NFR-05	Maintainability	All game logic shall stay inside the Snake class. Adding a new theme shall only require adding a new option to the enum and one new case in the drawing code.
NFR-06	Reliability	The game shall close cleanly at any point. No crashes or unexpected behavior shall happen when the snake dies, hits a border, or when keys are pressed quickly.

NFR#	Category	Requirement
NFR-07	Scalability	The grid size and cell size shall be set using named constants so the game can be made bigger or smaller by changing just one number.
NFR-08	Responsiveness	The game shall check for key presses every frame so no input is missed between movement steps.

2.6 Scope

2.6.1 In Scope

- A fully playable Snake game on a 20x20 grid controlled by keyboard.
- All screens: main menu, wall selection, snake theme, fruit type, ready screen, gameplay, pause, and game over.
- Two wall modes: wrap-around and classic wall death.
- Four snake color themes: Classic Green, Colorful Rainbow, Ocean Blue, Sunset Orange.
- Four fruit types: Apple, Orange, Blueberry, Golden.
- Session high score shown in the top bar and on the game over screen.
- Speed increases automatically as the player scores more points.
- All graphics drawn using basic SFML shapes and text, no image files needed.

2.6.2 Out of Scope

- Saving the high score after the game closes.
- Playing with another person or over a network.
- Running on phones, browsers, or non-Windows computers.
- Using a gamepad, mouse, or touch screen.
- Sound effects or music.
- Using image files or animated graphics.
- An AI that plays the game automatically.

2.7 Limitations

#	Limitation	Impact
L01	The high score is only saved while the game is open. It disappears when the game closes.	Players cannot track their best score over multiple sessions.

#	Limitation	Impact
L02	The game keeps getting faster with no maximum speed limit.	At very high scores, the game becomes too fast to control.
L03	The font file arial.ttf must be in the same folder as the game.	If it is missing, the game will not start at all.
L04	The snake's eyes are always drawn in the same spot on the head, no matter which direction it is moving.	The eyes do not turn to face the direction of movement.
L05	The game only works with a keyboard.	Players without a keyboard cannot play.
L06	There is no way to manually set the difficulty. Speed only changes based on score.	Beginners cannot slow the game down.

Chapter 3: Use Case Model

3.1 Actors

The system has one actor:

- Player — a person who uses the keyboard to go through the menus and play the game.

3.2 Use Case Diagram Description

The game has five main use cases that the player can trigger through the keyboard.

UC-01: Navigate Main Menu and Customization

Field	Details
Use Case ID	UC-01
Name	Navigate Main Menu and Customization
Actor	Player
Trigger	Player opens the game. The main menu appears.
Preconditions	The game is running and arial.ttf is in the same folder as the game.
Main Flow	1. The main menu appears with the title and a blinking prompt. 2. Player presses ENTER. 3. The wall mode screen appears. 4. Player presses W or D. 5. The snake color screen appears. 6. Player presses 1 to 4. 7. The fruit type screen appears. 8. Player

Field	Details
	presses 1 to 4. 9. The ready screen shows a summary of all choices. 10. Player presses any key. 11. The game starts.
Alternate Flow	If the player closes the window at any screen, the game exits cleanly.
Postconditions	The game starts with the player's chosen settings.
Related FRs	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06

UC-02: Play Game

Field	Details
Use Case ID	UC-02
Name	Play Game
Actor	Player
Trigger	Player finishes the setup screens and starts the game.
Preconditions	The game is set up. The snake starts at the center of the grid with 3 segments.
Main Flow	1. The grid, snake, and food are drawn every frame. 2. The snake moves one step at a time in the current direction. 3. Player presses arrow keys to change direction. 4. Reverse direction inputs are ignored. 5. When the snake eats food, it grows, the score goes up, and new food appears. 6. The speed increases with the score. 7. The game continues until the snake dies or the player pauses.
Alternate Flow	Player presses P to pause. Snake hits itself or a wall and the game ends.
Postconditions	Score increases as food is eaten. The top bar always shows the current score and high score.
Related FRs	FR-06, FR-07, FR-08, FR-09, FR-10, FR-11, FR-12, FR-15, FR-16

UC-03: Pause and Resume Game

Field	Details
Use Case ID	UC-03
Name	Pause and Resume Game
Actor	Player

Field	Details
Trigger	Player presses P during gameplay.
Preconditions	The game is currently being played.
Main Flow	1. The game pauses. 2. A dark overlay appears with the word PAUSED and a hint to press P again. 3. Player presses P. 4. The game resumes exactly where it stopped.
Alternate Flow	Player closes the window and the game exits cleanly.
Postconditions	The game continues from the exact point it was paused. Nothing moves while paused.
Related FRs	FR-13

UC-04: Wrap-Around Wall Mode

Field	Details
Use Case ID	UC-04
Name	Wrap-Around Wall Mode
Actor	Player
Trigger	Player chose Wrap-Around mode and the snake reaches the edge of the grid.
Preconditions	The game is running and wrap-around mode is turned on.
Main Flow	1. The snake reaches the edge of the grid. 2. The game moves the snake to the opposite side. 3. The game continues without stopping.
Alternate Flow	None — this always works without failure.
Postconditions	The snake keeps moving from the other side of the grid.
Related FRs	FR-11

UC-05: Game Over and Replay

Field	Details
Use Case ID	UC-05
Name	Game Over and Replay
Actor	Player

Field	Details
Trigger	The snake runs into itself, or into a wall in Classic mode.
Preconditions	The game is currently being played.
Main Flow	1. The game detects that the snake has died. 2. If the score is higher than the saved high score, the high score is updated. 3. The game over screen appears with the final score and high score. 4. If the player beat the high score, it says NEW HIGH SCORE! 5. A blinking message tells the player to press ENTER to play again. 6. Player presses ENTER and goes back to the wall selection screen.
Alternate Flow	Player closes the window and the game exits cleanly.
Postconditions	The high score is updated if the player beat it. The player can start a new round with new settings.
Related FRs	FR-12, FR-14

Chapter 4: System Design Overview

4.1 OOP Architecture

The game is designed using Object-Oriented Programming. The Snake class holds all the game data and handles all game logic. The main() function manages the window, keeps track of which screen is showing, and draws the UI, but it passes the actual game work to the Snake class.

Two helper functions — drawSelScreen() and drawReadyScreen() — are used to draw the setup screens. They take the screen content as input instead of having it hard-coded, which makes it easy to add new screens later. New color themes or fruit types can be added by simply adding a new option to the list and writing one new drawing case.

Class / Component	Type	What It Does
Snake	Class	Holds the snake body, food position, settings, and cell size. Handles movement, collision, food, and drawing.
SnakeTheme	Enum	Lists the four snake color options: GREEN, COLORFUL, BLUE, ORANGE. Used to pick colors when drawing the snake.
FruitTheme	Enum	Lists the four fruit options: APPLE, ORANGE_FRUIT, BLUEBERRY, GOLDEN. Used to pick the food color.

Class / Component	Type	What It Does
main() / State Machine	Driver	Keeps track of which screen is active. Owns the window, font, clock, and score. Passes settings to Snake when a game starts.
drawSelScreen()	Helper Function	Draws any selection screen. Takes a title and a list of options as input. Used for wall, snake, and fruit screens.
drawReadyScreen()	Helper Function	Draws the summary screen before the game starts, showing the player's chosen settings.

4.2 Game Architecture and State Machine

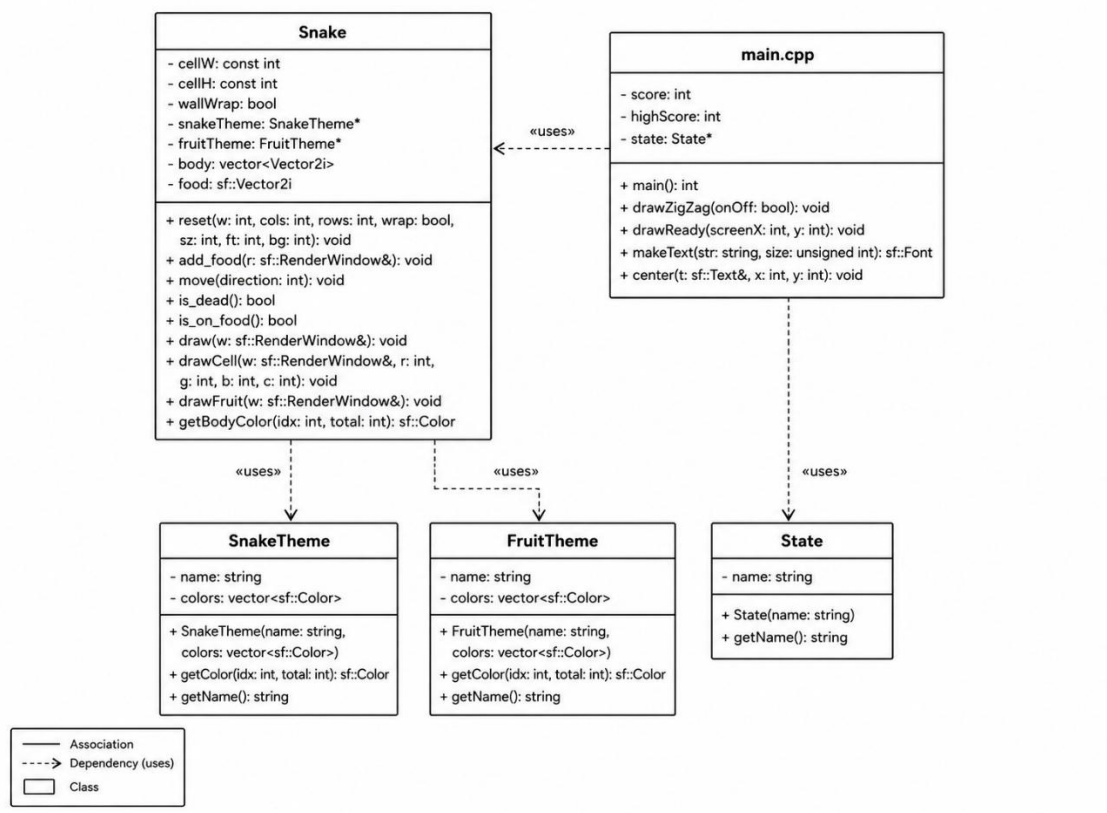
The game loop runs at 60 frames per second. The snake does not move every frame — it moves on a timer. The timer starts at 200 milliseconds and gets 3 milliseconds shorter for every point scored, making the game gradually faster. A separate timer controls the blinking effect on prompts, toggling every 500 milliseconds.

The game is always in one of eight states. Each state has clear rules for how to enter and exit it.

State	How You Get In	How You Get Out
MENU	Game is launched	Player presses ENTER → goes to SEL_WALL
SEL_WALL	From MENU or GAMEOVER	W key → SEL_SNAKE with wrap on. D key → SEL_SNAKE with wrap off
SEL_SNAKE	From SEL_WALL	Keys 1 to 4 → goes to SEL_FRUIT with chosen theme
SEL_FRUIT	From SEL_SNAKE	Keys 1 to 4 → goes to READY with chosen fruit
READY	From SEL_FRUIT	Any key → game starts, goes to PLAYING
PLAYING	From READY	P key → PAUSED. Snake dies → GAMEOVER
PAUSED	From PLAYING	P key → back to PLAYING
GAMEOVER	Snake dies in PLAYING	ENTER key → back to SEL_WALL

4.3 UML Class Diagram Description

The key connections between classes of the system are as follows:



Chapter 5: SDLC Model

5.1 Selected Model: Incremental Model

The team used the Incremental Development Model for this project. This means the game was built in stages. Each stage added new features on top of a working version from the previous stage. This allowed the team to test and fix each part before moving on to the next.

5.2 Justification

The Incremental Model was chosen for these reasons:

- The game naturally breaks into separate layers: core engine, game rules, visual themes, and full UI. Each layer can be built and tested on its own.
- After the first stage, a basic but playable Snake game already exists. This gives the team early feedback.

- The Snake class was designed from the start to accept theme settings later, so adding themes in stage 3 did not require changing the existing structure.
- Each stage can be tested on its own before the next one begins, which reduces bugs and makes fixing easier.

5.3 Development Increments

Increment	What Was Delivered	Features Added	Files Changed
Increment 1	Core Game Engine	Snake class, 20x20 grid, food spawning, movement, collision detection, score tracking, basic top bar.	Snake.h, Snake.cpp, main.cpp
Increment 2	Wall Mode and Speed	Wrap-around wall mode, score-based speed increase, direction buffer to stop 180-degree turns.	Snake.cpp, main.cpp
Increment 3	Themes and Customization	Snake color themes, fruit color themes, gradient body drawing, snake eyes.	Snake.h, Snake.cpp
Increment 4	Full UI Flow	Main menu, all selection screens, ready screen, game over screen, pause overlay, blinking effects.	main.cpp

Chapter 6: Assumptions and Constraints

6.1 Assumptions

- The game will run on a 64-bit Windows computer. The SFML DLL files and arial.ttf will be in the same folder as the game.
- The screen is at least 640 pixels wide and 640 pixels tall.
- The player has a keyboard with arrow keys and number keys.
- The compiler supports C++17 or newer, which is needed for some features used in the menu code.
- The SFML 3.x version is used. The older SFML 2.x version will not work with this code.
- Basic random number generation is enough for placing food. High-quality randomness is not needed.

6.2 Constraints

- The game shall only use C++ and SFML 3.x. No other libraries are allowed.
- The window size is fixed at 600 pixels wide and 640 pixels tall. It cannot be resized while the game is running.
- The high score is only kept while the game is open. No files or databases are used to save it.
- The grid is always 20 columns and 20 rows. The snake always starts with 3 segments in the center.
- The game speed has a minimum limit of 70 milliseconds per step so it never becomes physically impossible to play.

Chapter 7: Project Risk Management

7.1 Overview

Risk management means finding problems that could affect the project before they happen and planning how to deal with them. For The Snake Within, planning ahead helps make sure the team can finish the game on time and without major issues.

7.2 Risk Identification and Projection

ID	Risk	Details	Category	Probability
T1	SFML 3.x API Changes	SFML 3.x works differently from SFML 2.x. Using the wrong documentation could cause errors that are hard to find.	Technical	0.4
T2	Font File Missing	If arial.ttf is not in the game folder, the game will not start and shows an error.	Technical	0.3
O1	Team Member Unavailable	If a developer who knows SFML or C++ becomes unavailable, parts of the project could be delayed.	Operational	0.3
Q1	Drawing Errors	If coordinates are calculated wrong, the snake or food might appear in the wrong place or overlap the top bar.	Quality	0.25
S1	UI Takes Longer Than Expected	Building and testing all the menu screens and screen transitions might take more time than planned.	Schedule	0.35

7.3 Risk Mitigation, Monitoring and Management (RMMM) Plan

RMMM T1: SFML 3.x API Changes

- Mitigation: All developers shall only use the official SFML 3.x documentation and the working project code as a reference. No code from SFML 2.x guides should be used without checking first.
- Monitoring: The team lead will review any new SFML code before it is added and test it immediately.
- Management: If something in SFML 3.x is unclear, the developer will write a small separate test program to check how it works before adding it to the main project.

RMMM T2: Font File Missing

- Mitigation: The arial.ttf file will always be included in the project folder. A checklist will be used before every test or submission to confirm all files are present.
- Monitoring: The error message shown when the font fails to load will be used as an indicator during testing.
- Management: If arial.ttf cannot be included for legal reasons, a free font such as Liberation Sans will be used instead and the file name will be updated in the code.

RMMM O1: Team Member Unavailable

- Mitigation: All team members will learn how the main parts of the code work. Code will be written with clear comments so anyone can pick it up.
- Monitoring: Progress and task ownership will be checked during regular team meetings.
- Management: If someone becomes unavailable, their tasks will be divided among the remaining team members and the instructor will be informed if there is a schedule impact.

RMMM S1: UI Takes Longer Than Expected

- Mitigation: Each screen will be built and tested one at a time before connecting them together. No screen will depend on another until both are working individually.
- Monitoring: After each screen is finished, a quick check will confirm it works before moving to the next.
- Management: If the UI work falls behind schedule, cosmetic details like exact colors and spacing will be skipped for now and finished later, while making sure all screens are functional for submission.

Chapter 8: Verification and Validation

8.1 Testing Strategy

The game will be tested by playing it and checking specific inputs and behaviors. Every test case is linked to one or more requirements to make sure everything that was specified has been checked.

8.2 Test Cases

TC#	What to Test	What Should Happen	Related FR
TC-01	Open the game with arial.ttf present.	Main menu shows the title and a blinking ENTER prompt. No errors.	FR-01
TC-02	Press ENTER → W → 1 → 1 → any key.	Each screen appears in order. Ready screen shows the correct choices. Game starts.	FR-02, FR-03, FR-04, FR-05
TC-03	Press ENTER → D → 2 → 2 → any key. Steer snake into a wall.	Snake dies when it hits the wall. Game over screen appears with score.	FR-10, FR-14
TC-04	Press ENTER → W → 1 → 1 → any key. Steer snake off the right edge.	Snake comes out from the left side and keeps moving.	FR-11
TC-05	During gameplay press P, then press P again.	Game pauses and shows the PAUSED overlay. Resumes when P is pressed again.	FR-13
TC-06	Eat 5 food items and watch the speed.	The snake moves noticeably faster than at the start.	FR-07, FR-12
TC-07	While moving right, press the Left arrow key.	Snake keeps moving right. Direction does not change.	FR-08
TC-08	Steer the snake into its own body.	Snake dies and the game over screen appears.	FR-10
TC-09	Get a high score in round 1. Start round 2 and check the top bar.	The high score from round 1 is still shown in round 2.	FR-12, FR-14
TC-10	Play with all 4 snake themes and all 4 fruit types in separate rounds.	Each theme shows different colors. Snake and food match the selected options.	FR-15, FR-16

8.3 Acceptance Criteria

The game is considered complete and acceptable when all of the following are true:

- All ten test cases pass without any errors or unexpected behavior.
- The game runs smoothly at 60 frames per second with no freezing or flickering.
- All menu screens are easy to understand and the player can complete the setup without confusion.
- No crash or freeze happens in any test scenario, even when keys are pressed quickly.
- The game runs correctly on a Windows computer with SFML 3.x files in the same folder.

Appendix A: Revision History

Version	Date	Author	What Changed
0.1	April 2026	Development Team	First draft. Scope and objectives written.
0.5	April 2026	Development Team	Requirements and stakeholder table added.
0.9	May 2026	Development Team	Use cases, system design, SDLC section, and test cases added.
1.0	May 2026	Development Team	Final review and formatting done. Document submitted.

Appendix B: Glossary of Terms

Term	What It Means
SFML	A C++ library that provides tools for graphics, window management, and input handling. Used for all drawing in this game.
sf::RenderWindow	The SFML object that represents the game window where everything is drawn.
sf::Clock	An SFML timer used to measure time between snake movement steps and control the blinking effect.
State Machine	A system where the program is always in one specific state such as MENU or PLAYING, and switches between states based on what the player does.
HUD	The top bar shown during gameplay that displays the score, high score, and pause hint. Stands for Heads-Up Display.
Grid Cell	One square on the 20x20 game grid. Each cell is 30x30 pixels on screen.
Wrap-Around	A wall mode where the snake comes out from the opposite side of the grid instead of dying when it hits an edge.

Term	What It Means
nextDir	A variable that stores the player's most recent key press and applies it on the next movement step. This prevents the snake from instantly reversing.
Frame Rate Limit	A setting that caps the game loop at 60 cycles per second to keep performance stable and avoid using too much CPU.